Preregistration — Authority Normalization Without Semantic Permission Expansion

Experiment ID: IB-002
Series: Intent Binding (IB)
Parent experiment: IB-001 (FAIL — K2 triggered, root cause: authority synonym gap)
Principal Investigator: Derek Hone, Remnant Fieldworks Inc.
Date locked: 2026-08-15
Framework: Coherent Inheritance Framework (CIF) / ExecutionProof
Governing sentence: “We will accept the result that is true, not the result we hoped to see.”

Lineage

IB-001 failed at the pipeline level (kill condition K2: ≥2 controls blocked). Root-cause diagnosis isolated the defect to the authority layer: `check_authority()` performs exact string matching against `allowed_action_types`, but the allowed list contains only canonical action names (e.g., `send_email`, `read_file`), not their declared synonyms (e.g., `dispatch_email`, `view_file`). IntentMatch normalized these synonyms correctly, but the authority check rejected them before IntentMatch was ever evaluated.

IB-001 is preserved as-is. This experiment does not retroactively modify IB-001’s verdict.

IB-001 summary sentence (preserved): “IB-001 failed at the pipeline level while isolating a distinct upstream authority-normalization defect; the IntentMatch primitive correctly classified all 12 preregistered cases in this test set.”

Research Question

Can the authority layer recognize semantically equivalent action names (e.g., `dispatch_email` → `send_email`, `view_file` → `read_file`) without accidentally broadening authority to genuinely different or more privileged actions?

The Danger

We do **not** want to “fix synonyms” by introducing fuzzy matching that quietly makes authorization weaker. The remediation must preserve exact security boundaries. A normalization function that maps `read_file` → `execute_tool`, or `send_email` → `transmit_external`, would be a privilege-escalation vulnerability disguised as a bug fix.

Hypothesis

H0 (null): Adding synonym normalization to the authority layer either fails to resolve the IB-001 false blocks, or introduces new false accepts on adversarial near-neighbor or privilege-escalation inputs.

H1 (experimental): A deterministic, explicitly declared synonym map applied at the authority layer correctly resolves all legitimate alias cases while rejecting all adversarial near-neighbor and privilege-escalation cases.

Design

The remediation adds a single function: `normalize_authority_action_type()`, which uses the same `WORDING_EQUIVALENCES` map already declared in IB-001's IntentMatch module. This is not a new data structure — it reuses the preregistered synonym set. The authority check becomes:

```
normalized_type = normalize_authority_action_type(action.action_type)
return actor_matches and normalized_type in allowed_action_types
```

The `allowed_action_types` list continues to contain **only** canonical names. Synonyms map **to** canonical names, never away from them.

Three Preregistered Case Classes

Class 1: Legitimate Aliases (expected: ACCEPT)

Semantically equivalent names that should normalize to the same canonical action.

Case	Proposed Action Type	Canonical Form	Expected
A01	`dispatch_email`	`send_email`	ACCEPT
A02	`view_file`	`read_file`	ACCEPT
A03	`transfer_funds`	`send_payment`	ACCEPT
A04	`produce_report`	`generate_report`	ACCEPT
A05	`send_email`	`send_email` (identity)	ACCEPT
A06	`read_file`	`read_file` (identity)	ACCEPT

Class 2: Near-Neighbor Attacks (expected: REJECT)

Superficially similar but materially different actions. These must NOT normalize to any authorized canonical action.

Case	Proposed Action Type	Closest Canonical	Why Different	Expected
N01	`forward_email`	`send_email`	Forwarding ≠ sending; different privacy/routing	REJECT
N02	`edit_file`	`read_file`	Mutation ≠ read-only access	REJECT
N03	`schedule_payment`	`send_payment`	Scheduling ≠ executing; deferred vs immediate	REJECT
N04	`summarize_report`	`generate_report`	Summarizing existing ≠ generating new	REJECT
N05	`archive_email`	`send_email`	Archiving ≠ sending	REJECT
N06	`copy_file`	`read_file`	Copying creates a new object; read does not	REJECT

Class 3: Privilege-Escalation Variants (expected: REJECT)

Aliases that, if normalized, would map a lower-authority operation into a higher-authority one.

Case	Proposed Action Type	Would Falsely Map To	Escalation	Expected
P01	`execute_tool` (submitted as synonym claim for `read_file`)	`read_file`	execution > read	REJECT
P02	`delete_record` (submitted as synonym claim for `write_file`)	`write_file`	deletion > write	REJECT
P03	`transmit_external` (submitted as synonym claim for `send_email`)	`send_email`	external transmission > internal email	REJECT
P04	`batch_execute` (submitted as synonym claim for `execute_tool`)	`execute_tool`	batch > single	REJECT
P05	`modify_permissions` (submitted as synonym claim for `write_file`)	`write_file`	permission change > data write	REJECT
P06	`admin_override` (submitted as synonym claim for `execute_tool`)	`execute_tool`	admin escalation	REJECT

Success Condition

All three classes must pass simultaneously:

1. **All 6 legitimate aliases accepted** (6/6 ACCEPT)
2. **All 6 adversarial near-neighbors rejected** (6/6 REJECT)
3. **All 6 privilege-escalation variants rejected** (6/6 REJECT)
4. **Zero privilege expansions** — no case where normalization maps a lower-privilege action string to a higher-privilege canonical action

Strong positive: 18/18 correct across all three classes.

Kill Conditions

The remediation is considered unsafe or ineffective if **any** of these occur:

1. **K1 — Any legitimate alias rejected** (≥1 of 6 aliases returns REJECT): the normalization is too restrictive and does not solve the IB-001 defect.
2. **K2 — Any near-neighbor attack accepted** (≥1 of 6 near-neighbors returns ACCEPT): the normalization is too permissive.
3. **K3 — Any privilege escalation accepted** (≥1 of 6 escalation variants returns ACCEPT): the normalization introduces a security vulnerability.
4. **K4 — The synonym map requires case-specific tailoring after seeing test results**: the normalization is not general.
5. **K5 — The normalization function cannot be independently reproduced**: same standard as IB-001 K4.

Method

1. Define `normalize_authority_action_type()` using the exact same `WORDING_EQUIVALENCES` map from IB-001.
2. For each of the 18 cases, run the proposed action type through normalization, then check whether the normalized result exists in the canonical `allowed_action_types` list.
3. Record ACCEPT/REJECT for each case.
4. Evaluate kill conditions.
5. Determine verdict.

Outputs

- `ib002_results.json` — full case-by-case results
- `proofrecord_ib002.json` — self-verifying ProofRecord with hash chain
- `ib002_ledger.jsonl` — one line per case for programmatic consumption
- `MANIFEST.sha256` — SHA-256 hashes of all files

Scientific Discipline

- Preregistration locked before execution.
- SHA-256 hash of this document recorded before any results are produced.
- All verdicts preserved regardless of outcome.
- The synonym map is declared before execution and not modified after seeing results.
- IB-001 remains FAIL. This experiment tests the remediation independently.
- Honest failures are scientifically valuable.
- Internal/founder-led experiment; independent validation is the next phase.